UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

IA PARA ENGENHEIROS
UFRGS

Curso de Especialização

Inteligência Artificial para Engenheiros

Análise de Dados e Engenharia de Características

Aula 06:

Codificação de Variáveis Categóricas e Engenharia de Atributos

Professor: **Vinicius Horn Cene**

2026

Direitos Autorais

Este material possui Direitos Autorais e Direitos de Imagem e não pode ser duplicado, reproduzido, no todo ou em parte, sob quaisquer formas ou por quaisquer meios (eletrônicos, mecânico, gravação, fotocópia, distribuição na internet ou outros), sem permissão expressa do autor.

©

Termo de Responsabilidade da Própria UFRGS

Todos os materiais disponibilizados nos Ambientes Virtuais da UFRGS são exclusivamente para usos e fins didáticos, sendo vedada a sua utilização para qualquer outra finalidade, sob as penas legais.

Todos os materiais de terceiros que venham a ser utilizados ou disponibilizados devem ser referenciados, sob as penas legais, inclusive de plágio.

Tenho liberdade de expor ou não a minha imagem e voz nas atividades, o que não me desonera da participação efetiva nas mesmas.

Todas as gravações de atividades síncronas devem ser previamente informadas por parte dos professores.

Os alunos somente poderão gravar, armazenar ou compartilhar atividades mediante concordância prévia do professor, sob as penas legais.

Os alunos somente poderão armazenar ou compartilhar materiais mediante concordância prévia do professor ou se permitido no material, sob as penas legais.

É proibido disponibilizar, por quaisquer meios digitais ou físicos, os dados, a imagem e a voz dos alunos, dos colegas e do professor, sem autorização específica para a finalidade pretendida.

Onde estamos no pipeline de ML

Raw Data

↓

EDA

↓

Data Cleaning / Cleansing

↓

Feature Engineering ← estamos aqui

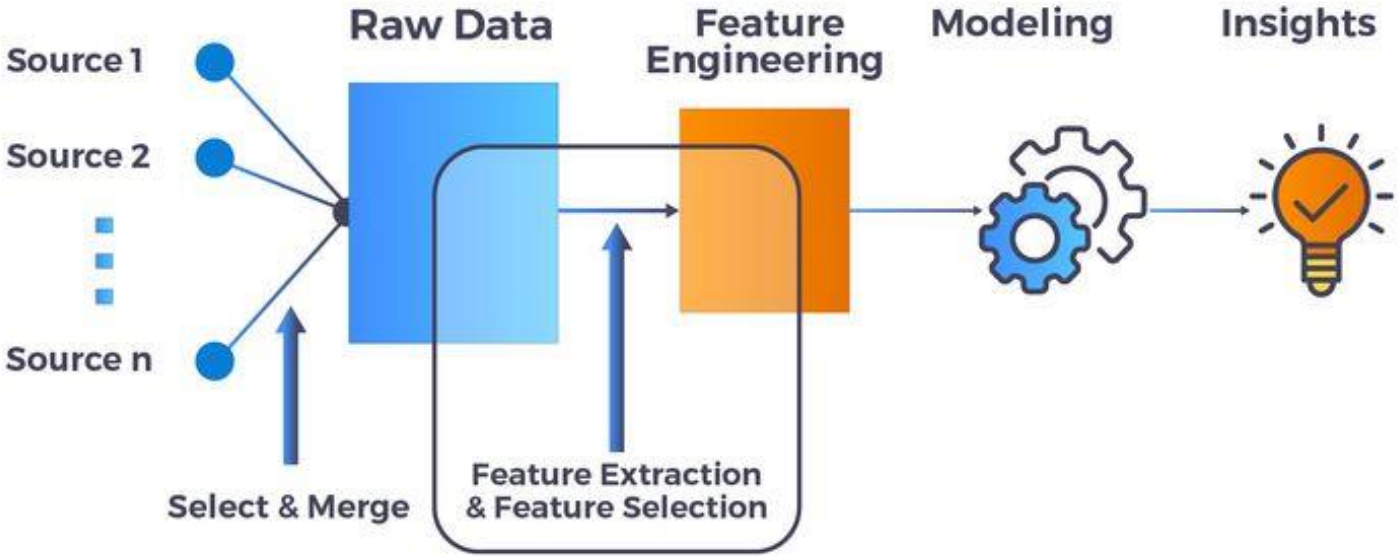
↓

Feature Selection

↓

Model Training

Feature Engineering



Fonte Imagem: ML pipeline com ênfase em feature engineering

Motivação

Problema central:

- Muitos datasets reais possuem **preditores categóricos**
- Algoritmos de ML **operam sobre números**
- Encoding inadequado **degrada performance**

Objetivos da aula (ao final da aula vocês deverão):

- entender **como representar variáveis categóricas**
- conhecer **diferentes técnicas de encoding**
- saber **quando usar cada técnica**
- criar **novas features úteis**
- entender **impacto no modelo**

O que é uma Feature?

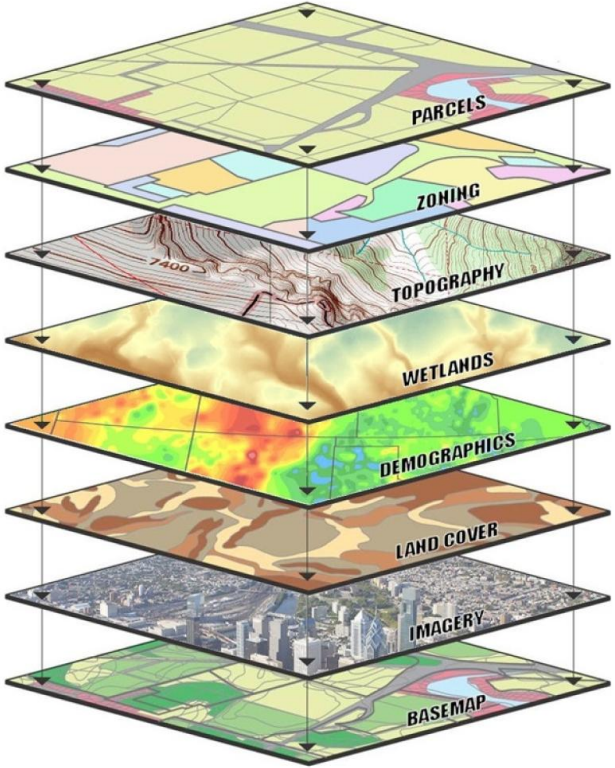
Feature: variável de entrada do modelo.

Tipos comuns:

- numéricas
- categóricas
- temporais
- texto
- geoespaciais

$$\begin{bmatrix} x_{1,1} & \cdots & x_{c,1} \\ \vdots & \ddots & \vdots \\ x_{1,l} & \cdots & x_{c,l} \end{bmatrix} = \begin{bmatrix} Y_1 \\ Y_2 \\ Y_3 \end{bmatrix}$$

$l: total\ de\ linhas$
$c: total\ de\ colunas$



***modelos trabalham em espaços vetoriais**

Fonte imagem: https://geocloudanalytics.com/wp-content/uploads/2022/11/What_Is_GIS-1.png

Tipos de variáveis categóricas

Nominal

Ordinal

Tipos de variáveis categóricas

Nominal

- cor
- cidade
- marca

Ordinal

- escolaridade
- classificação de risco
- nível de satisfação

- **nominal ≠ ordinal (ranking)**

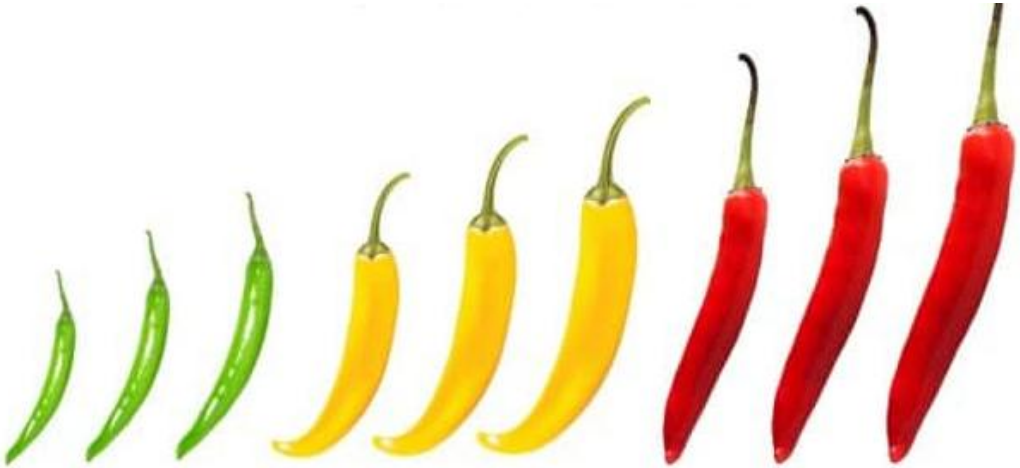
Problema fundamental

Cores

- verde
- amarelo
- vermelho

Se codificados como:

- Verde = 1
- Amarelo = 2
- Vermelho = 3



- **Temos que:** Vermelho > Amarelo > Verde **(o que não faz sentido)**

Estratégias de Encoding

- **Label Encoding:** mapeia categorias para inteiros
- **One-Hot Encoding:** categorias viram **vetores adicionais binários**
- **Ordinal Encoding:** quando **ordem real existe**
- **Frequency Encoding:** baseada na **frequência da categoria**.
- **Target Encoding:** substituir categoria pela **média do target**
- **Binary Encoding: combinação** de **label** encoding e **representação binária**
- **Embeddings:** categorias viram vetores que capturam **similaridade entre elas** (usado principalmente em Deep Learning e alta cardinalidade)

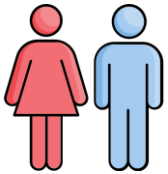
Estratégias de Encoding

"Encoding não é só transformar dados, é escolher como o modelo vai enxergar o mundo"

Label Encoding

- **Converte categorias** em **ints**
- Cada **categoria** recebe um **ID único**

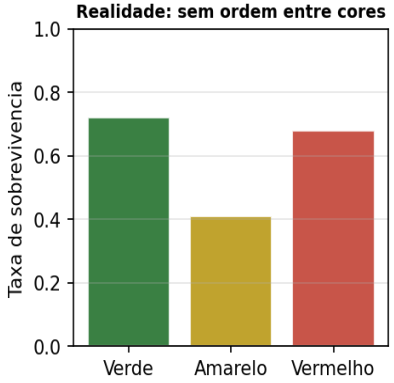
- **Sexo:**
  Male → 0
  Female → 1

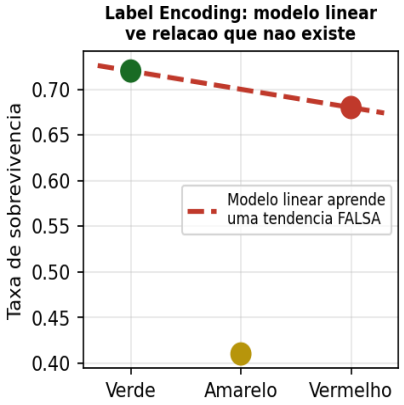


- **Problema principal:**
  O modelo pode interpretar **ordem que não existe (alto > médio > baixo)**

Quando isso é um problema?

- Modelos lineares ($\hat{y} = 0.3 \times \text{cor} + f_2 + f_3 + \cdots$)
- ANN,
- KNN

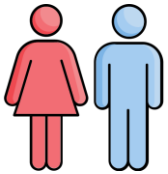




Label Encoding

- **Converte categorias** em **ints**
- Cada **categoria** recebe um **ID único**

- **Sexo:**
  Male → 0
  Female → 1



- **Problema principal:**
  O modelo pode interpretar **ordem que não existe (alto > médio > baixo)**

Quando isso é um problema?

- Modelos lineares ($\hat{y} = 0.3 \times \text{cor} + f_2 + f_3 + \cdots$)
- ANN,
- KNN

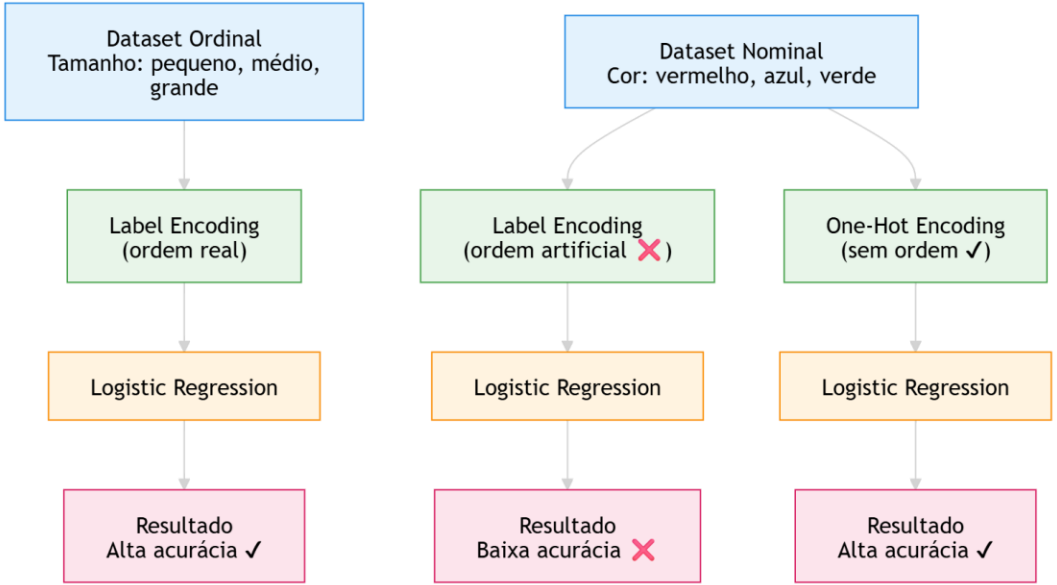
Quando pode funcionar bem?

- Modelos de árvores (Random Forest, Decision Trees)
- Quando a variável é **ordinal de verdade**

Label Encoding

🎯 **Regra prática sobre Label Encoding:**

- ❌ **Não usar** para categorias nominais (cores, marcas, etc...)
- ✅ Usar apenas quando houver **ordem real**



One-Hot Encoding

- **Converte categorias** em **múltiplas colunas binárias**
- Cada **categoria** vira uma **feature independente (0 ou 1)**

- **Cor:**
  Branco, Verde, Amarelo

$$\begin{array}{ccc} B & V & A \\ \end{array}$$
$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

- **Vantagem principal:**
  Representa categorias **sem introduzir ordem artificial**

...	Marca	Preço
...	Volkswagen	30.000
...	Toyota	35.000
...	Honda	37.000
...	Honda	41.000

...	Volkswagen	Toyota	Honda	Preço
...	1	0	0	30.000
...	0	1	0	35.000
...	0	0	1	37.000
...	0	0	1	41.000

Quando isso é um problema?

- Alta cardinalidade (muitas categorias)
- Dados ficam esparsos (muitos zeros)
- Alto custo computacional
- Pode causar overfitting

Quando pode funcionar bem?

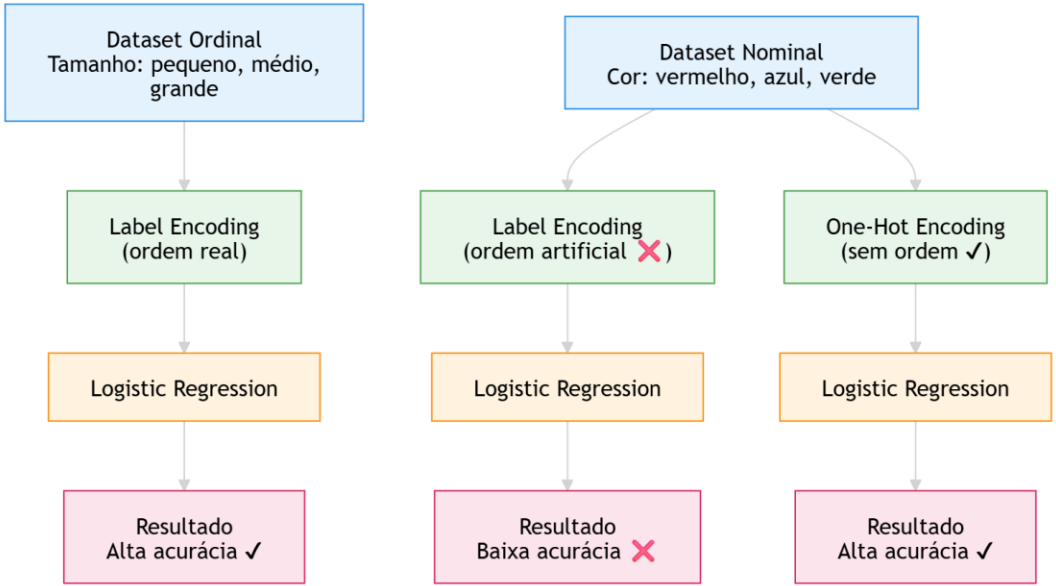
- **Baixa** cardinalidade
- Categorias **nominais** (sem ordem)
- Modelos **lineares** (LR), ANN, KNN
- Situações onde **interpretabilidade** é importante

One-Hot Encoding

🎯 **Regra prática sobre One-Hot Encoding:**

✅ **Usar** para categorias nominais (cores, marcas, etc...)

❌ Usar apenas quando houver **ordem real**



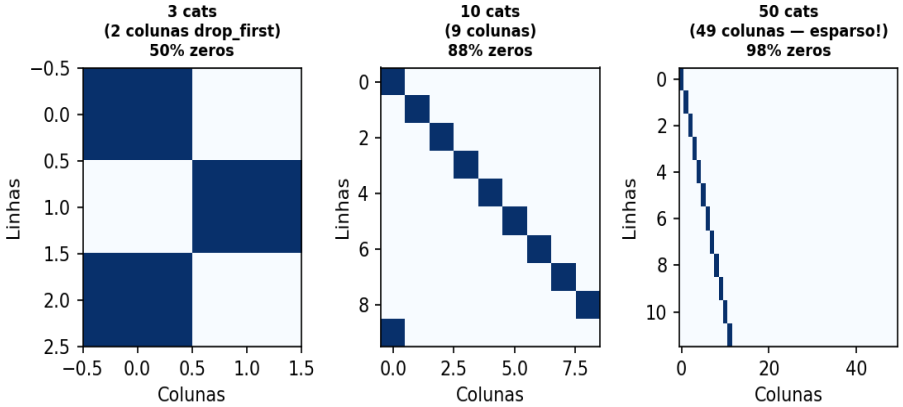
One-Hot Encoding: Esparsidade e Dimensionalidade

- 3 categorias: 2 colunas (drop_first) - 10 categorias: 9 colunas - 50: 49 colunas
- Com 1000 categorias: 999 colunas, cada linha com 998 zeros — 99.9% de zeros
- Matrizes esparsas: scipy.sparse — armazena só os 1s para economizar memória
- KNN sofre especialmente: distância euclidiana em espaço esparso e quase constante
- **Diagnóstico: se após o OHE, temos mais features que amostras, há risco de overfitting!**

drop_first=True:

- **A categoria removida é a referencia:** seus coeficientes são zero por definição

- **Interpretação:** coeficiente de 'Amarelo' = diferença em relação à 'Verde' (referência)



- **Escolha da referência:** a mais frequente ou a de baseline do negócio (seja explícito)
- **Nunca:** escolha automática sem documentar qual categoria foi removida

Label vs One-Hot Encoding

🎯 Pode causar explosão de dimensionalidade (1000 categorias → 1000 colunas)

"Se Label Encoding cria ordem demais...

One-Hot cria dimensão demais"

Ordinal Encoding

- **Converte categorias** em **inteiros (respeitando uma ordem real)**
- Cada **categoria** recebe um **valor que representa a sua posição**

- **Nível de risco:** Baixo (0), Médio (1), Alto (2)

- **Vantagem principal:**
  Preservar **relação de ordem entre categorias**

- **Desvantagem principal:**
  Distância entre valores pode **não refletir a realidade**
  (ex: Médio **não é necessariamente** “o dobro” de Baixo)

Quando isso é um problema?

- **Não existe ordem real** entre categorias
- Pode introduzir **ordem artificial (igual ao Label Encoding)**
- Modelos **podem** interpretar **diferenças** como “**distância**”

Quando pode funcionar bem?

- Variáveis **naturalmente ordinais**: Tamanho (P, M, G), Risco (B, M, A), etc
- **Modelos lineares e árvores**

Ordinal Encoding

🎯 **Em suma:**

"Ordinal Encoding é basicamente o Label Encoding... só que feito com consciência"

- **Label Encoding** → ❌ perigoso
- **One-Hot** → ✓ seguro, mas computacionalmente caro
- **Ordinal** → ✓ correto, quando faz sentido

Frequency Encoding

- Substitui **categoria** pela sua **frequência no dataset**
- Representa o **quão comum** aquela categoria é

- **Cidades:**
  SP (50%), RJ (30%), POA (20%)

SP → 0.5
RJ → 0.3
POA → 0.2

- **Vantagem principal:**
  Reduz dimensionalidade e funciona bem com **alta cardinalidade**

- **Desvantagem principal:**
  Frequência ≠ importância para o modelo (categoria **comum ≠** categoria **mais relevante**)

Quando isso é um problema?

- Desconsidera **relação** com o **target**
- **Categorias diferentes** podem ter a **mesma frequência**
- Pode **perder informação** relevante
- Pode introduzir **interpretação numérica indevida**

Quando pode funcionar bem?

- Alta cardinalidade (**muitas classes**)
- **Modelos** que lidam bem com **variáveis numéricas**
- Como **baseline rápido**

df['cat'].value_counts(normalize=True)

Frequency Encoding

🎯 Frequency Encoding é rápido e barato…
mas meio ‘cego’.

“Se Frequency Encoding ignora o target...o próximo encoding faz exatamente o contrário”

Target Encoding

- Substitui cada **categoria** pela **média do target**
- **Informação supervisionada** para criar a feature

- **Taxa de Venda de X Produto:**
  SP (50%), RJ (30%), POA (20%)

  SP → 0.5
  RJ → 0.3
  POA → 0.2

- **Vantagem principal:**
  Captura diretamente a **relação com o target**

- **Desvantagem principal:**
  Pode "**ensinar**" o modelo com **informação** do **próprio target** (**vazamento de informação / data leakage**)

Quando isso é um problema?

- Alto **risco de overfitting**
- **Sensível a poucas observações**
- Pode introduzir **vazamento de informação**

Quando pode funcionar bem?

- Alta cardinalidade (**muitas classes**)
- Problemas supervisionados
- Quando feito corretamente (com validação / folds)

Target Encoding - Data Leakage

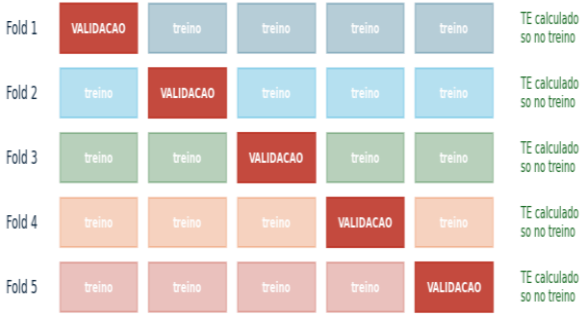
- **Problema crítico: O encoding pode usar informação do próprio target**

- **Exemplo:**
  Cidade → média usando **TODO** o dataset
  - **Cada linha usa informação de si mesma**
  - **Isso é data leakage**

- **Consequências:**
  - **Performance artificialmente alta**
  - **Modelo não generaliza**
  - **Resultados irreais em produção**

- **Intuição:**
  - **O modelo está "vendo a resposta" durante o treino**

- **Como corrigir?**
  - Calcular encoding **apenas no treino**
  - Usar ***cross-validation*** (***K-Fold***)
  - **Técnicas como:**
    - ***Leave-One-Out Encoding*** (LOO)
    - **Target Encoding com *smoothing***



Target Encoding

🎯 Se você não tomar cuidado…
Target Encoding vira “trapaça estatística”

“Target Encoding é poderoso porque usa o target… e perigoso pelo mesmo motivo:

Se você fizer errado, o modelo não aprende, ele memoriza”

Frequency vs Target Encoding: Comparativo

- **Frequency Encoding — use quando:**
  - Não ha supervisão disponível (treino não-supervisionado)
  - A frequência tem significado de negocio (cidades populosas vs raras)
  - Velocidade e critica — calculado instantaneamente sem risco de leakage

- **Target Encoding — use quando:**
  - A relação entre a categoria e o target é o que importa
  - Cardinalidade alta torna OHE inviável
  - É possível garantir OOF dentro de um Pipeline correto

Binary Encoding

- Converte **categorias** em **números inteiros**
- Representa esses **ints em formato binário**
- **Cada bit** vira uma **nova coluna**

- **Categorias: A, B, C, D**

Label Encoding:

A → 0
B → 1
C → 2
D → 3

Binário:

A → 0
B → 1
C → 2
D → 3

col_1	col_2 :
0	0
0	1
1	0
1	1

- **Desvantagem - Eficiência x Interpretação:**
  Não tão expressivo quanto **One-Hot**
  Nem tão simples quanto **Label** encoding

Quando isso é um problema?

- Introduzir **relações artificiais entre bits**
- **Menos interpretável**
- **Não captura relação** com o **target**
- Pode introduzir **vazamento de informação**

Quando pode funcionar bem?

- Alta cardinalidade (**muitas classes**)
- Quando **One-Hot é inviável**
- **Modelos baseados em árvore**
- **Necessidade: solução** mais **compacta**

- **Vantagem principal:**
  Reduz dimensionalidade comparado ao One-Hot encoding

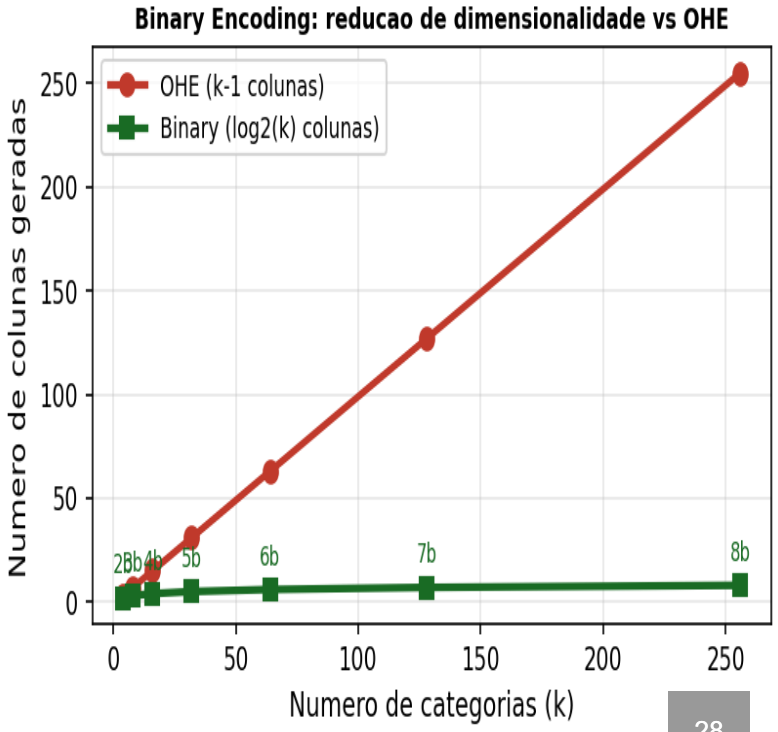
Binary Encoding

- Converte **categorias** em **números inteiros**
- Representa esses **ints em formato binário**
- **Cada bit** vira uma **nova coluna**

- **Categorias: A, B, C, D**

Label Encoding:	**Binário:**	**col_1**	**col_2 :**
A → 0	A → 0	0	0
B → 1	B → 1	0	1
C → 2	C → 2	1	0
D → 3	D → 3	1	1

- **Desvantagem - Eficiência x Interpretação:**
  Não tão expressivo quanto **One-Hot**
  Nem tão simples quanto **Label** encoding



Binary Encoding

🎯 **Em suma:**

"Binary Encoding é uma compressão do One-Hot... com algumas consequências"

"Encoding é sempre um trade-off entre informação, custo e risco"

- **Label** → simples, mas perigoso
- **One-Hot** → correto, mas caro
- **Binary** → eficiente, mas com trade-offs

Embeddings

- **Categorias como vetores** numéricos **densos**
- **Vetores aprendidos automaticamente** pelo modelo
- Capturam **relações e similaridades entre categorias**

- **Categorias:**
  Gato, Cachorro, Avião

Gato → [0.2, 0.8, 0.1]
Cachorro → [0.3, 0.7, 0.2]
Avião → [0.9, 0.1, 0.8]

- **Vantagem principal:**
  Captura **similaridade semântica** entre categorias

- **Desvantagem principal:**
  Depende do modelo aprender bem os vetores
  (**se mal treinado, perde significado**)

Quando isso é um problema?

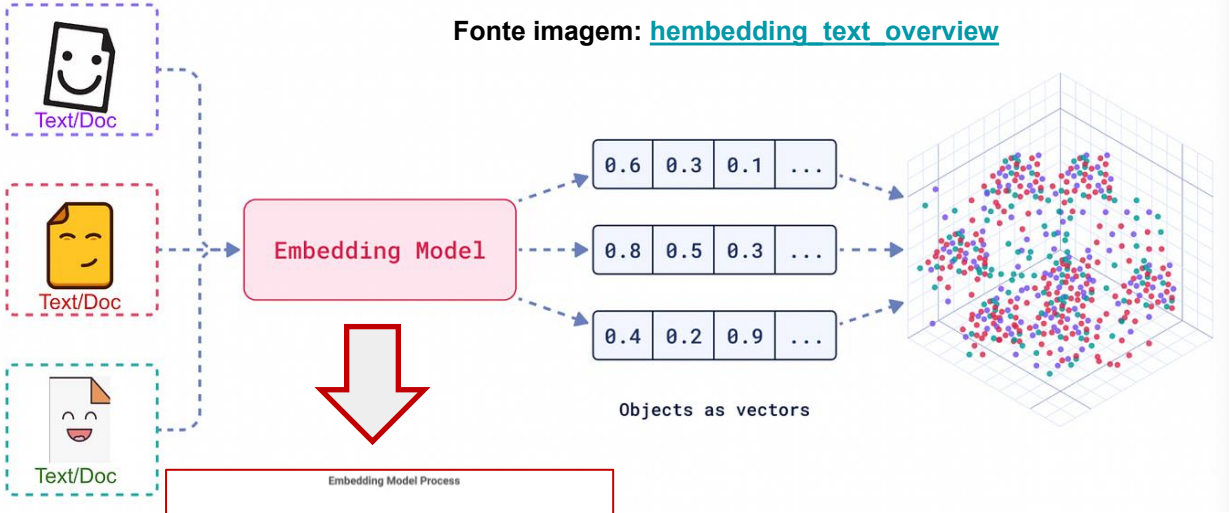
- Mais **complexo de implementar**
- **Menos interpretável**
- **Requer mais dados** (DL)
- **Overfitting se mal treinado**

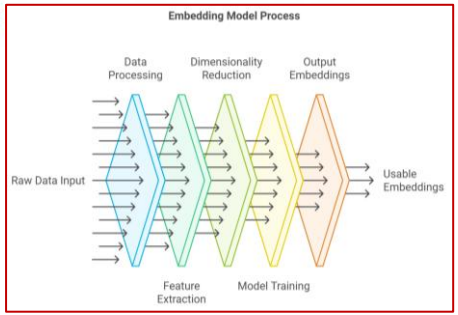
Quando pode funcionar bem?

- Alta cardinalidade (**milhares de categorias**)
- **Deep Learning / redes neurais**
- **Sistemas de recomendação**
- **NLP e problemas complexos**

Embeddings

Fonte imagem: hembedding_text_overview





"Embedding não é só encoding, é representação aprendida."

Fonte imagem: Embedding_inner_view

Encodings

🎯 **“Não existe encoding perfeito, apenas o mais adequado para o contexto”**

Método	Ideia Principal	Vantagem	Problema Principal	Quando Usar
Label Encoding	Categoria → inteiro	Simples e rápido	Introduz ordem artificial	Raramente (ou árvores)
One-Hot Encoding	Categoria → vetor binário	Representação correta	Explosão de dimensionalidade	Baixa cardinalidade
Ordinal Encoding	Inteiros com ordem real	Preserva hierarquia	Distâncias podem ser irreais	Variáveis ordinais
Frequency Encoding	Usa frequência da categoria	Compacto e eficiente	Ignora o target	Alta cardinalidade
Target Encoding	Média do target por categoria	Captura relação com o target	Data leakage / overfitting	Alta cardinalidade (com cuidado)
Binary Encoding	Inteiro → binário	Reduz dimensionalidade	Menos interpretável	Alternativa ao One-Hot
“Embeddings”	Vetores aprendidos pelo modelo	Captura similaridade	Complexidade / baixa interpretabilidade	Deep Learning / muitos dados

Perguntas chaves:

- Existe ordem real?
- Quantas categorias existem?
- O modelo é linear ou árvore?
- Existe risco de leakage?

“Encoding não é pré-processamento… é uma decisão de modelagem.”

Feature Engineering

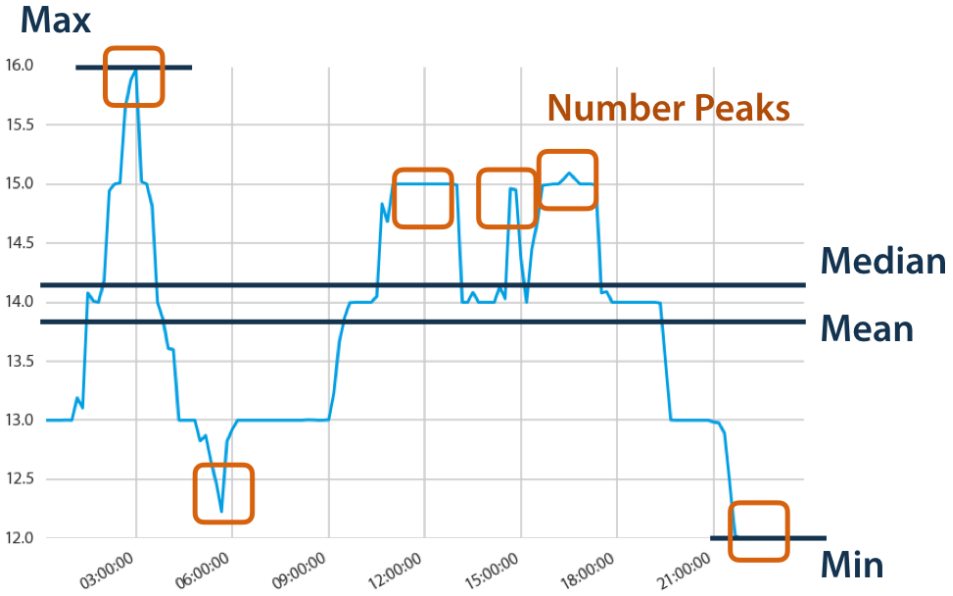
- Processo de criação e transformação de variáveis
  - Sinal informação
  - Informação requer semântica
  - Maximiza informação extraída do sinal
- Melhora a capacidade de aprendizagem do modelo
  - Diminui overfit
  - Treina mais rápido
  - Diminui curse of dimensionality
- Melhores features > modelos mais complexos
  - Possibilita a utilização de modelos mais simples
    - ➢ Economia de tempo
    - ➢ Economia de processamento
    - ➢ Economia de memória

Fonte Imagem: 1623226884583 (743×742)

Motivação para a criação de features

- Ideia central: criar **variáveis mais informativas que as originais**
- Modelos simples podem performar melhor se features forem descritivas o suficiente

- **Exemplo:**
  - **Séries Temporais**



Fonte Imagem: Time Series :: Part-3 :: Feature Engineering | by The Data Beast | Medium

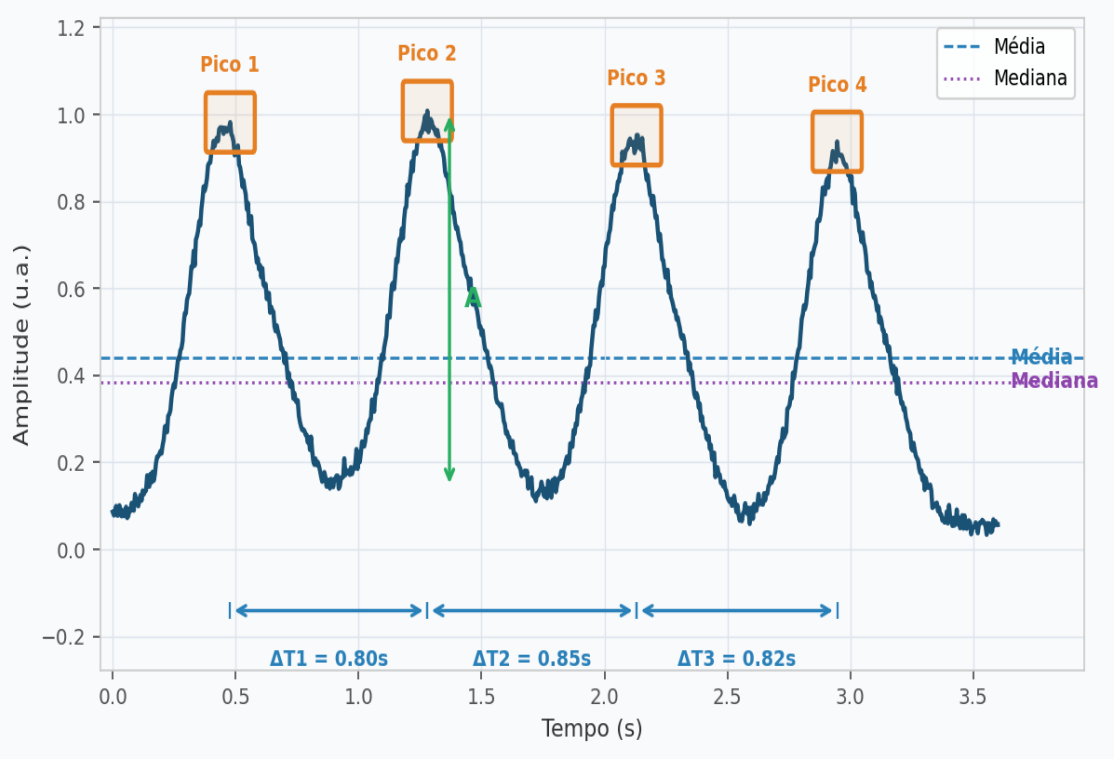
Motivação para a criação de features

- **Ideia central:**
  - criar variáveis mais informativas que o sinal bruto
  - Modelos simples com boas features superam modelos complexos

- **Exemplo: Sinal PPG**
  - Mede variação do volume sanguíneo via luz infravermelha

- **Features extraídas do sinal:**
  - Picos sistólicos: posição e Vmax
  - Intervalo ΔT entre picos consecutivos
  - Amplitude A: Vmax - Vmin
  - Média e Mediana do sinal

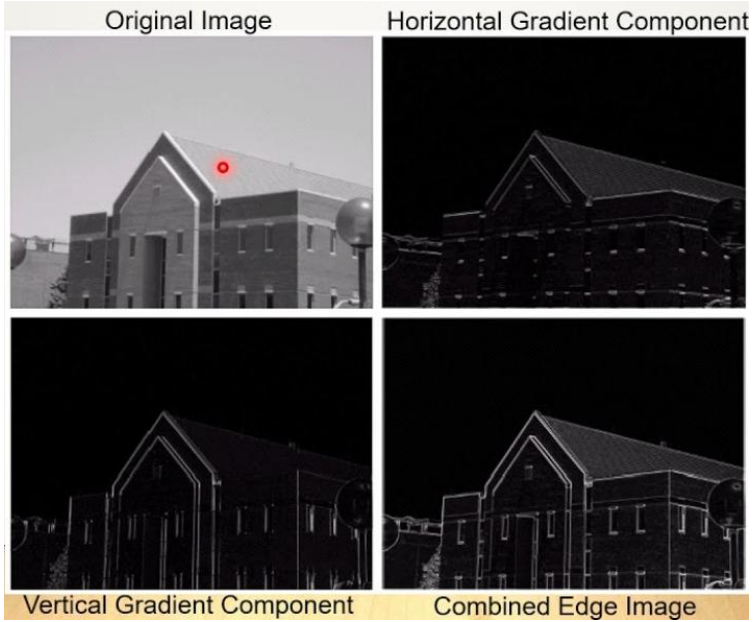
- **FC = 60 / média(ΔT) → ≈ 72 BPM**



Motivação para a criação de features

- Ideia central: criar **variáveis mais informativas que as originais**
- Modelos simples podem performar melhor se features forem descritivas o suficiente

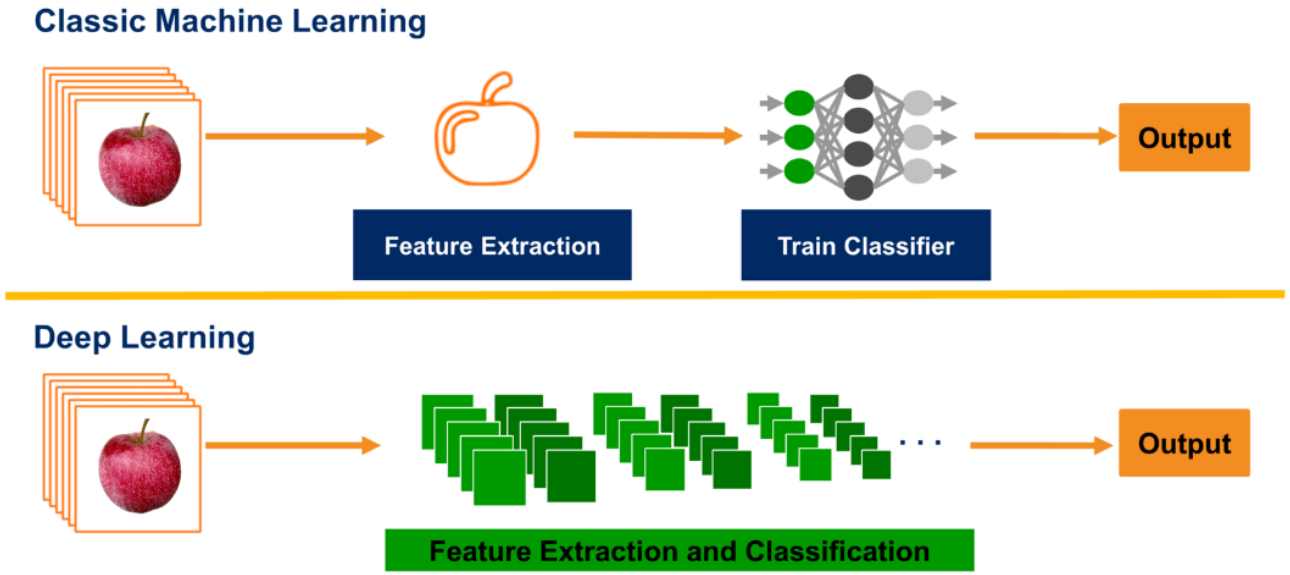
- **Exemplo:**
  - **Processamento de Digital de Imagens**



Fonte Imagem: https://www.youtube.com/watch?v=web4XCbsEXk

Motivação para a criação de features

- Ideia central: criar **variáveis mais informativas que as originais**
- Modelos simples podem performar melhor se features forem descritivas o suficiente
- **ML x DL:**



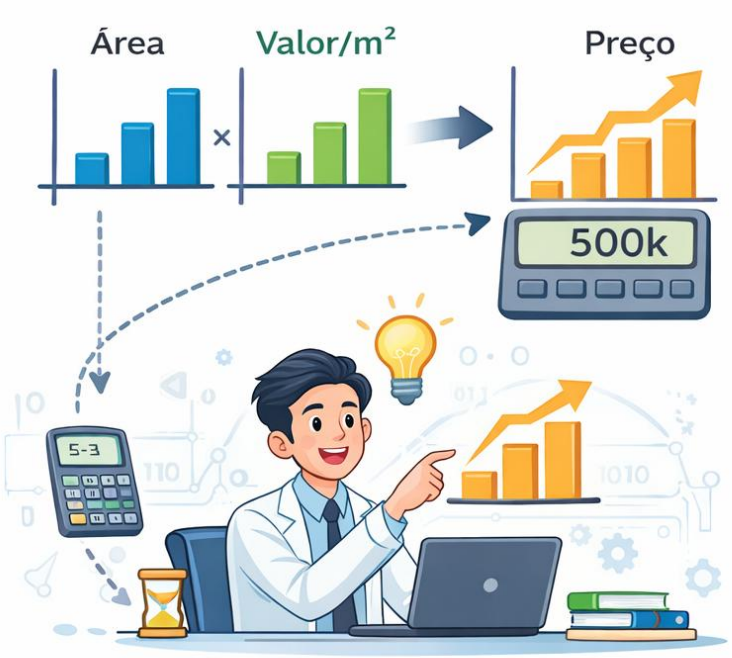
Fonte Imagem: https://kambale.dev/feature-extraction-in-ml

Estratégias de criação de features

- Interações
- Polinômios
- Agregações
- Transformações
- Utilização de domínio específico

Interações

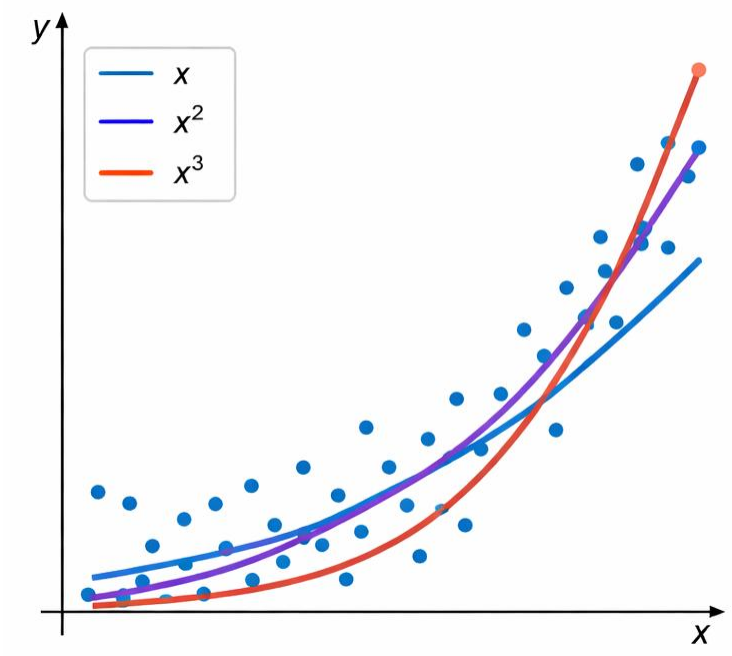
- Combinação de variáveis
- Útil para modelos lineares
- **Atenção:** risco de overfitting
  - Interações aumentam o poder do modelo, mas também o risco de decorar o dataset
  - Padrões que só existem no treino
  - Explosão de features
  - Explosão de features
  - Modelo fica sensível a ruído
- Usar poucas interações (sentido)
  - *área × preço_m²* → **faz sentido**
  - *idade × CEP* → **provavelmente não**



"Mais features não significa melhor modelo, significa mais risco."

Polinômios

- Criação de termos não lineares
- Ex: x^2, x^3
- Captura relações complexas
- **Atenção:** crescimento de features
  - Adicionar x^2 parece simples, mas abre a porta para uma explosão de variáveis
  - Ex. com combinações (2 feats → 5 feats):
    - X = [x1, x2] → [x1, x2, $x1^2$, $x2^2$, x1*x2]
  - Overfitting
  - Custo computacional
  - Instabilidade
  - Interpretabilidade desaparece



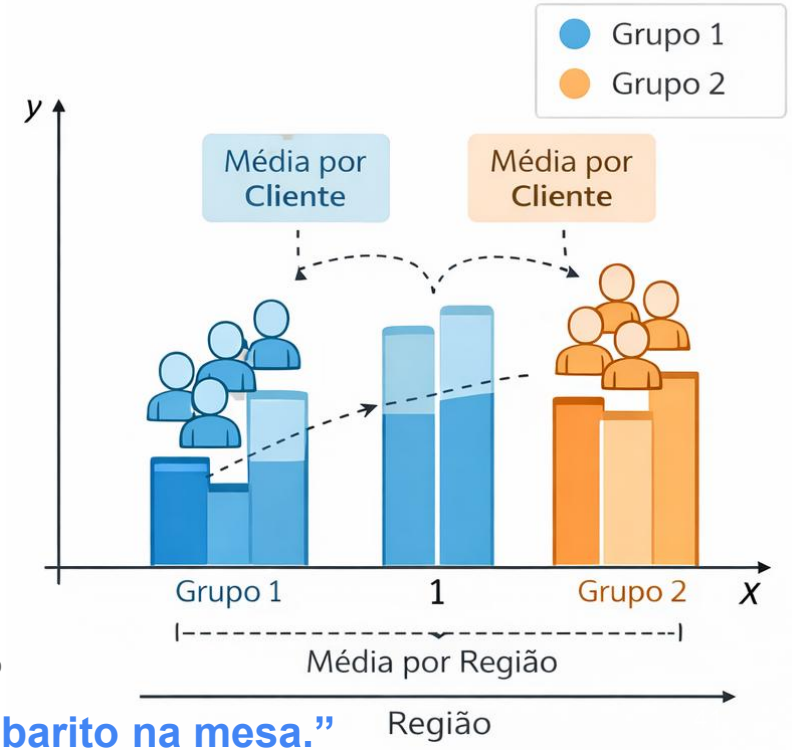
"Polinômios transformam modelos simples em altamente complexos."

Agregações

- Resumo estatístico de grupos
- Ex: média por cliente
- Muito usado em dados reais
- **Atenção:** data leakage

```
media_gasto_por_cliente=df.groupby("cliente")["gasto"].mean()
```

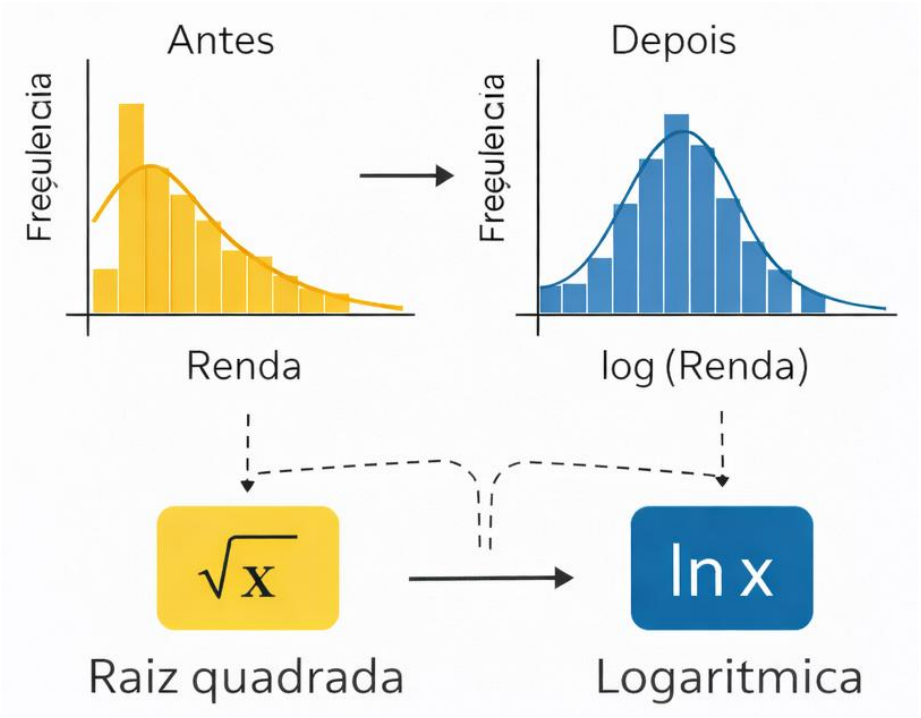
  - Se ***df*** conter treino + teste:
    - Estaremos usando dados de teste para criar features no treino
    - O modelo “vê o futuro”
    - Aprende padrões irreais
    - Performance explode (falsamente)
    - **Ex. Performance:** Treino=95% / Teste=60%



“É como estudar para uma prova com o gabarito na mesa.”

Transformações

- Alterações matemáticas
- Ex: log, sqrt
- Corrige distribuição
- **Atenção:** interpretação
  - Coeficientes perdem interpretação direta
  - Comparações ficam não intuitivas
  - Importância da variável pode enganar
  - Dificulta comunicação com stakeholders



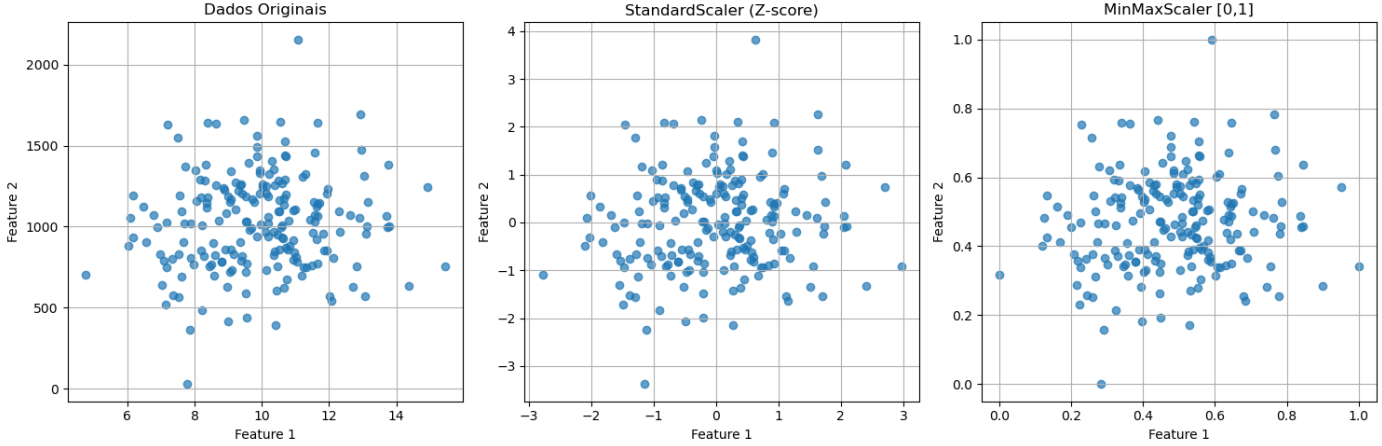
"Transformar melhora o modelo — mas pode piorar a interpretação."

Scaling

- Normalização de variáveis
- Ex: StandardScaler, MinMax
- Importante para KNN, LR, NN
- Desnecessário para árvores

- **KNN, LR, NN**
- Baseados em distância ou otimização de gradiente
- Sensíveis à escala das variáveis

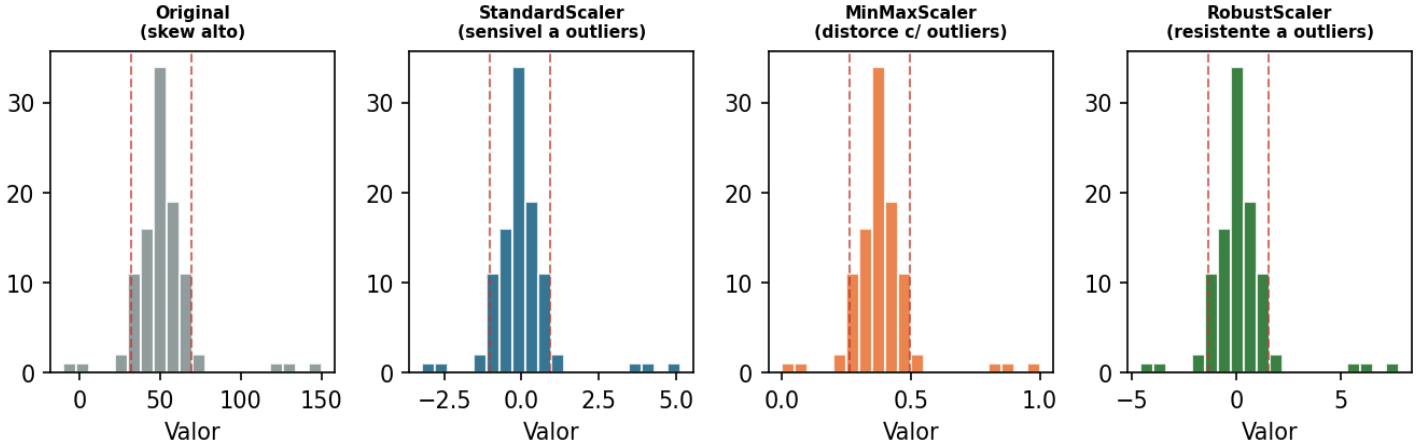
- **Desnecessário para árvores**
- Baseadas em regras (thresholds)
- Não dependem da magnitude dos dados



Robust Scaling

- **StandardScaler** (dados aproximadamente normais, modelos lineares): **z = (x - media) / std**
- **Problema:** outliers distorcem a media e std — ex: salario de CEO contamina todos

- **MinMaxScaler** (redes neurais -> ativações, imagens): **x_norm = (x - min) / (max - min)**
- **Problema:** extremamente sensível a outliers — um outlier extremo achatara os outros



Robust Scaling

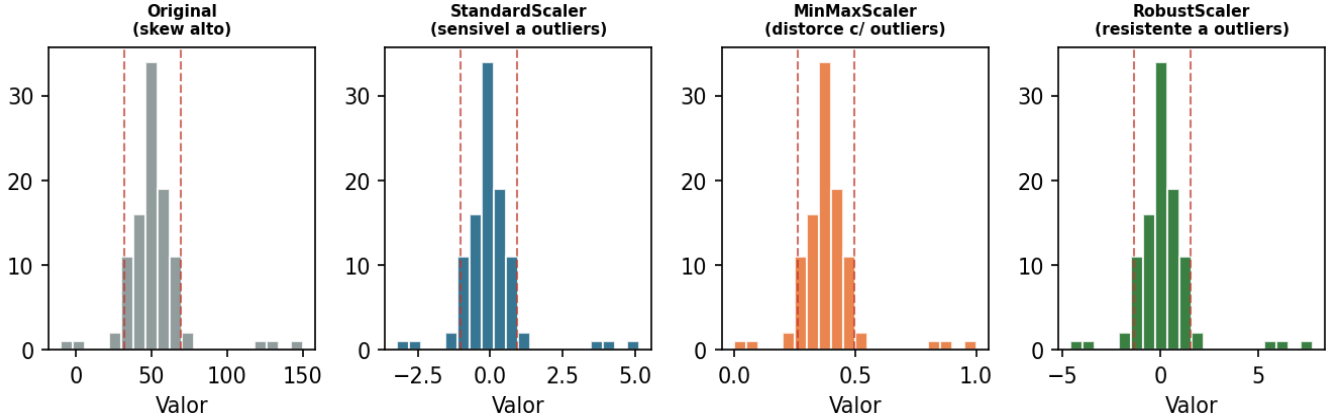
- **RobustScaler:** usa mediana e IQR em vez de media e std

  z = (x - mediana) / IQR onde IQR = Q75 - Q25

- **Vantagem:** outliers extremos não afetam os parametrons do scaler

  mediana e IQR são robustos a spikes — escala estável

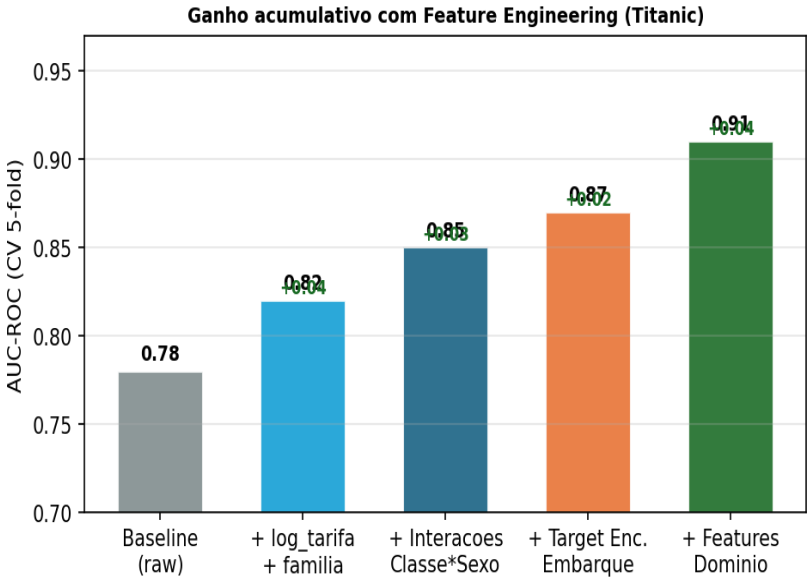
- **Quando usar:** dados industriais, financeiros ou medicos com outliers legítimos



Impacto Real: Ganho Acumulativo de Feature Engineering

- **Experimento:**
  - **Dataset:** Titanic
  - **Modelo:** Ridge Regressor como classificador
  - **Métrica:** AUC-ROC (CV 5-fold)

- **Resultados:**
  - **Baseline** (raw features): **AUC = 0.78**
  - + **log_tarifa** + **tamanho_familia**: **AUC = 0.82 (+0.04)**
  - + **interacao** classe x sexo: **AUC = 0.85 (+0.03)**
  - + **Target Encoding embarque**: **AUC = 0.87 (+0.02)**
  - + **features de domínio** (titulo, sozinho, classe_luxo): **AUC = 0.91 (+0.04)**

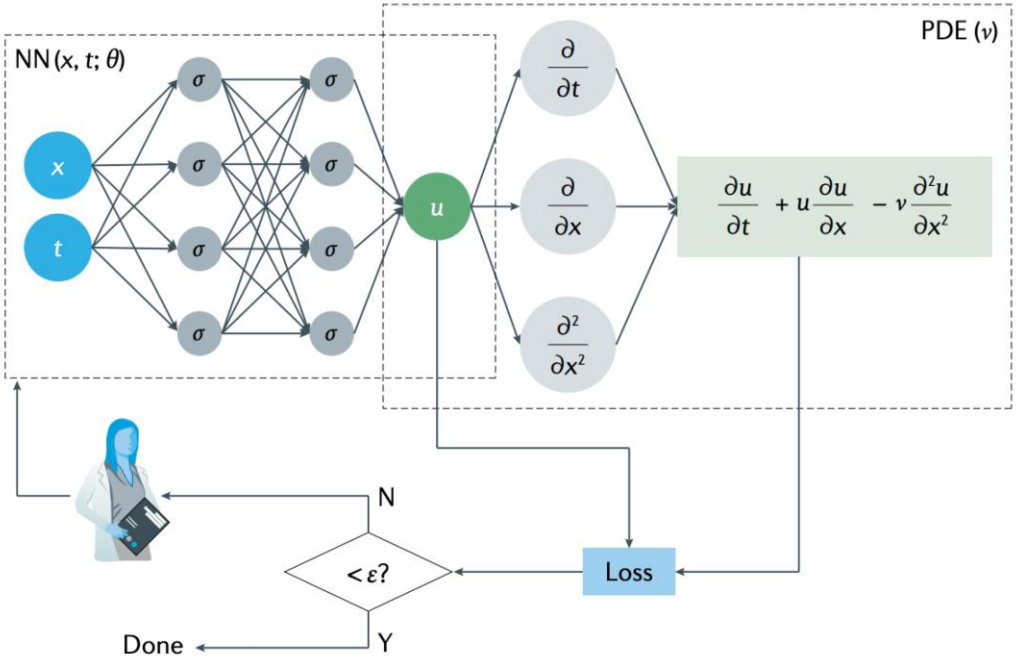


- **Conclusão:** ganhos incrementais acumulam — cada feature adiciona seu sinal único

Domínio Específico

- Uso de conhecimento do problema
- Muito poderoso em produção
- Depende de expertise
- Geralmente features específicas
- Também pode se apoiar em modelos específicos ex: PINNs

Fonte Imagem: Exemplo - Physics-Informed Neural Network



Feature Learning

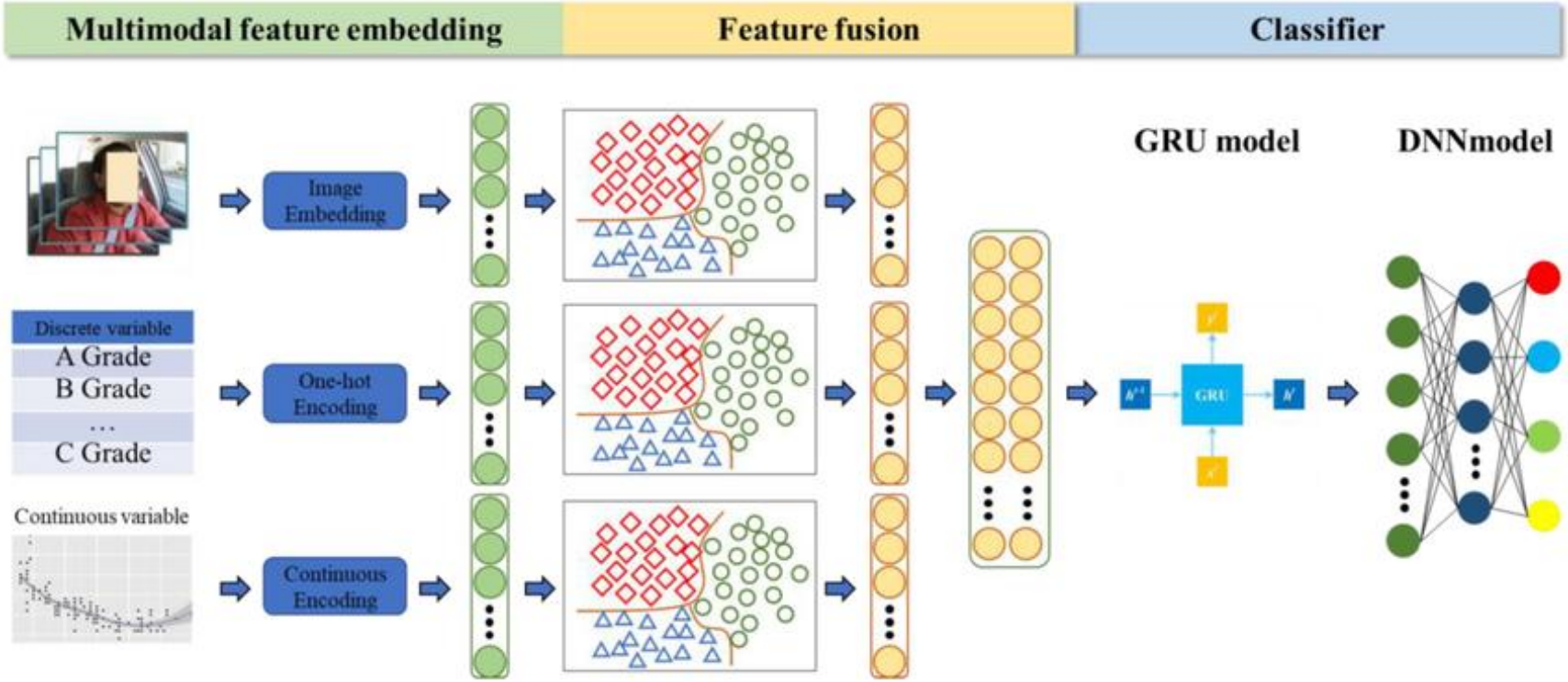
- Modelo aprende as features automaticamente
- Muito utilizado em Deep Learning
- Requer grande volume de dados para aprender com qualidade

- **CNNs:** aprendem filtros de bordas, texturas, formas diretamente dos pixels
- **BERT/transformers:** aprendem embeddings contextuais — 'banco' (financeiro) vs banco (movel)
- **Autoencoders:** comprimem features em representação latente densa — não-supervisionada

- **Quando usar:** dados não-estruturados (imagem, texto, áudio) com grande volume
- **Trade-off:** Feature Learning requer muito mais dados e computação que FE manual
- **Para dados tabulares com < 100k amostras:** FE manual geralmente > Feature Learning

Feature Learning

FONTE IMAGEM: Multimodal-feature-embedding



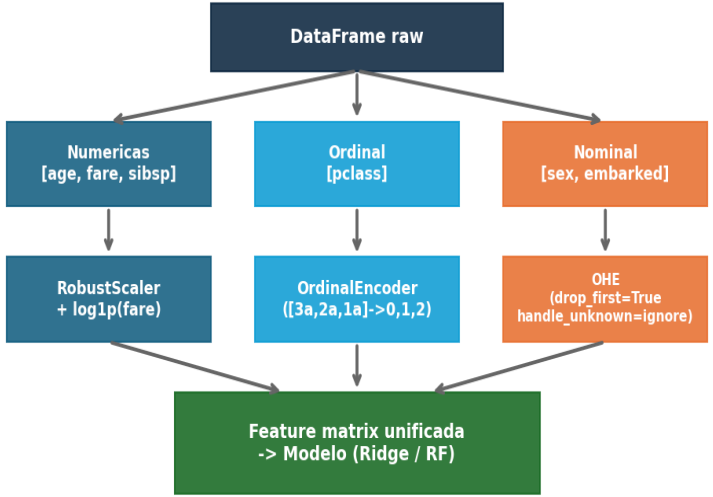
Estratégias de criação de features

Método	Ideia Principal	Vantagem	Problema Principal	Quando Usar	Quando NÃO usar
Interações	Combinação entre variáveis (ex: x1 × x2)	Captura relações não lineares simples	Risco de overfitting	Modelos lineares / relações entre variáveis	Poucos dados / muitas combinações
Polinômios	Criação de termos não lineares (x^2, x^3)	Aumenta capacidade do modelo	Explosão de features	Relações curvas / modelos simples	Alta dimensionalidade / risco de overfitting
Agregações	Estatísticas por grupo (média, soma, etc.)	Muito poder preditivo	Data leakage	Dados com grupos (clientes, regiões)	Sem controle temporal ou validação correta
Transformações	Aplicações matemáticas (log, sqrt, etc.)	Corrige distribuição / outliers	Perda de interpretabilidade	Dados skewed / modelos sensíveis à escala	Quando interpretação é crítica
Domínio Específico	Uso de conhecimento do problema	Alto ganho de performance	Depende de expertise	Projetos reais / produção	Quando não há entendimento do domínio

Pipeline Completo: Encoding + Scaling + FE

- **Um Pipeline de pré-processamento profissional encadeia:**

  1. Imputação de valores ausentes (por tipo de coluna)
  2. Criação de features de domínio
  3. Encoding categórico (OHE, Ordinal, TE — por coluna)
  4. Scaling numérico (RobustScaler ou StandardScaler)
  5. Seleção de features (SelectFromModel ou RFECV)
  6. Modelo final



Prof. Dr. Vinícius Horn Cene

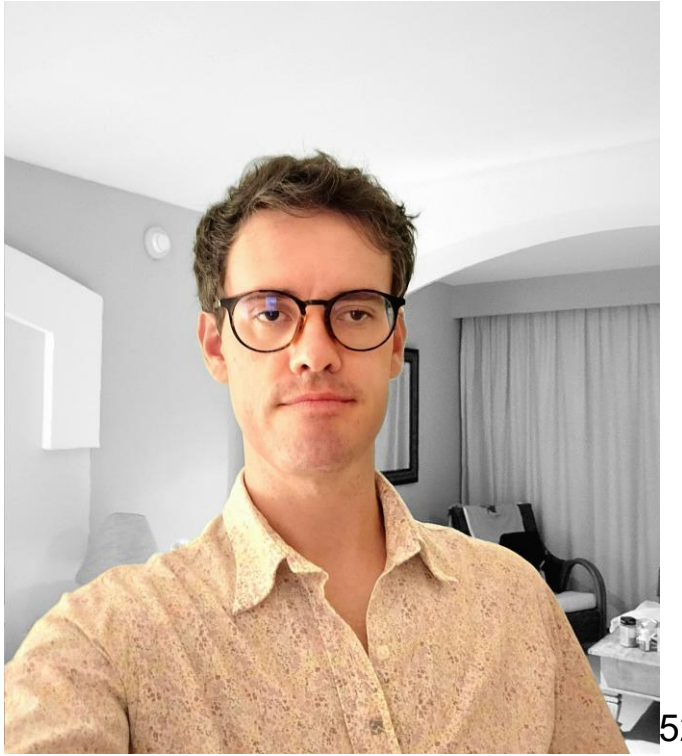
Departamento de Engenharia Elétrica

DELET – UFRGS

Prédio do Instituto Eletrotécnico

Sala #206-D

e-mail: vinicius.cene@gmail.com

Linked-In: https://www.linkedin.com/in/vinicius-cene-17695225/

Research Gate: https://www.researchgate.net/profile/Vinicius_Cene